

OlistOps Agent 项目介绍与技术报告

本地可运行的企业运营分析 Agent · v0.2.0

数据、RAG、API、工作流、部署、审计与技术面试说明

版式约定：正文中文使用宋体五号（10.5 pt）；英文、数字使用 Times New Roman 10.5 pt。报告内图片均为本机当前项目的真实页面或实时结果截图。

文档项	内容
项目目录	E:\Data\OlistOps_Agent
证据采集时间	2026-07-25 09:43:44 +0800
实时 Agent Run ID	d38715fa-9bf0-47e5-933d-7e48313efb8d
实时运行状态	completed
数据库	PostgreSQL 16.14
报告生成方式	同一结构化内容生成 DOCX 与 HTML，再由 Edge 打印为 PDF

阅读建议：业务或产品人员先看“项目定位、企业数据与典型场景”；开发人员重点看“系统架构、API、RAG、工作流”；技术面试官可直接查看“实现细节、质量验证、安全边界与面试问题”。

目录与快速结论

章节	主要回答的问题
1. 项目定位	它解决什么问题，当前做到什么程度？
2. 企业数据	用了什么数据，规模多大，怎样查看，怎样保证口径？
3. 系统架构	前端、API、Agent、工具、数据库、模型如何协作？
4. Agent 能力	支持哪些业务问题，如何规划和调用工具？
5. RAG	如何切块、建向量、混合检索、引用和评测？
6. API	客户端如何连接，会话、异步运行、SSE、Trace 怎样使用？
7. 真实运行	一次请求实际走过哪些节点，有哪些可审计证据？
8. 质量与安全	测试、评测、只读约束、注入防护和限制是什么？
9. 部署运维	从零启动、健康检查、重建数据与知识库怎么做？
10. 面试说明	技术取舍、追问与可扩展方向如何回答？

快速结论：项目已经具备可运行的 LangGraph 多阶段工作流、10 个受控分析工具、7 个 MART、PostgreSQL FTS + pgvector + RRF 混合 RAG、Ollama 本地模型、Critic、SSE 与 Trace。它是完整的本地工程样例，但不是已经具备租户、鉴权、审批恢复和高可用队列的生产 SaaS。

1. 项目定位与业务价值

1.1 一句话定位

OlistOps Agent 是一个面向电商运营人员的“证据驱动分析 Agent”。用户用自然语言提出订单、销售、履约、卖家、评价、支付、区域或运费问题；系统先把问题归类并生成计划，再调用白名单中的只读分析工具获取结构化指标，同时从企业知识库检索制度和操作规范，最后合成带证据引用的报告并执行质量审查。

1.2 为什么不是普通聊天机器人

普通问答风险	OlistOps 的工程控制
模型凭记忆给数字	经营数字只能来自参数化 SQL 与 MART 工具结果
模型任意生成 SQL	Agent 无自由 SQL 工具，只能调用注册表中的只读工具
建议缺少制度依据	RAG 检索知识片段，并生成 K-001 等可核对引用
执行过程不可追踪	记录 Run、Step、Tool Call、Event 和 Checkpoint
输出格式不稳定	Evidence Gate + Critic 校验非空、结构、引用覆盖和泄漏
外部模型不可用	Ollama 可本地推理，并保留确定性模板降级路径

1.3 当前实现边界

- 已实现：数据下载与装载、RAW/STAGING/MART、数据质量报告、10 个分析工具、10 类意图、9 节点 workflow、混合 RAG、Critic、REST/SSE/Trace API 和 React UI。
- 已实现：本地 PostgreSQL 16.14、pgvector、Ollama qwen2.5:3b，以及无需 API Key 的 local_hash 384 维向量回退。
- 尚未实现为生产级：正式登录鉴权、租户隔离、细粒度文档 ACL、TLS、可靠任务队列、分布式限流、审批恢复、完整可观测平台和自动扩缩容。
- 正式业务使用前仍需替换模拟制度文档、接入组织权限与真实 SLA，并由数据责任人确认指标口径。



图 1 当前本地 React 场景工作台。截图同时展示数据库、模型、RAG 状态和可提问场景。

2. 使用了什么企业数据，以及数据怎样看

2.1 数据来源与合规定位

项目使用 Brazilian E-Commerce Public Dataset by Olist（Olist 巴西电商公开数据集）。它是经过匿名化处理的公开电商交易数据，覆盖订单、订单商品、支付、评价、客户、卖家、商品、地理位置和品类翻译。项目把它作为“企业级关系数据建模与运营分析”的训练数据，不代表 Olist 当前生产系统，也不得用于重新识别个人或商户。

重要区分：交易数据是真实公开、已匿名化的数据；data/knowledge_seed 中的制度、SOP 和报告规范是为本项目构造的模拟企业知识，用于演示 RAG 与证据引用，不应被描述为 Olist 公司的真实内部政策。

2.2 当前本地数据库的真实规模

RAW 表	当前行数	业务含义
raw.customers	99,441	匿名客户与城市/州
raw.geolocation	1,000,163	邮编前缀与经纬度
raw.orders	99,441	订单状态和关键时间戳
raw.order_items	112,650	订单商品、卖家、价格、运费
raw.order_payments	103,886	支付方式、分期和金额
raw.order_reviews	99,224	评分、标题、文本和时间
raw.products	32,951	商品品类和物理属性
raw.sellers	3,095	匿名卖家与地区
raw.category_translation	71	品类葡英翻译

当前订单购买时间范围为 2016-09-04 至 2018-10-17。这是本次报告直接查询 PostgreSQL 得出的范围；不是手工写入报告的静态数字。



图 2 当前 PostgreSQL 实例的 RAW 与 MART 行数实时快照。

2.3 RAW、STAGING、MART 三层怎样分工

层	职责	代表对象	为什么这样设计
RAW	保留源 CSV 的原始字段和粒度，原则上只读	raw.orders、raw.order_items	可追溯、可重复装载，防止清洗逻辑污染源数据
STAGING	类型转换、时间字段解析、基础清洗和统一命名	staging.stg_orders	隔离脏数据与分析模型，集中定义字段语义
MART	按订单、卖家、品类、评价、支付和月份预聚合	mart.mart_order_fulfillment 等	避免 Agent 临时拼接复杂 JOIN，稳定粒度和指标口径

MART	当前行数	主要用途
mart_order_fulfillment	99,441	订单履约
mart_seller_order_fulfillment	100,010	卖家-订单履约，消除多商品重复
mart_category_order_fulfillment	99,470	品类-订单履约
mart_review_analysis_base	99,224	评价分析
mart_payment_analysis_base	101,686	支付分析
mart_seller_performance_monthly	16,441	卖家月度表现
mart_category_performance_monthly	1,283	品类月度表现

2.4 数据怎么查看

1. 最直观：浏览器打开 data/processed/data_quality_report.html，查看行数、主键重复、孤儿记录、时间缺失和数值范围。
2. 用 Swagger：访问 http://127.0.0.1:8000/docs，展开 analytics 接口并输入日期、group_by、top_k 等参数。
3. 用 Agent UI：访问 http://127.0.0.1:5173，用自然语言提问，再打开 Trace 看工具参数和返回摘要。
4. 用 SQL：只查询 mart 或 staging 层；排查源数据时再对 raw 做只读查询。
5. 用 API：适合 Notebook、BI 中间层或其他服务，通过 JSON 获取结构化结果。

```
# PowerShell: 直接查看一个 MART 的前 10 行
& ".\runtime\pgsql\bin\psql.exe" `
    "postgresql://olistops:olistops@127.0.0.1:55432/olistops" `
    -c "SELECT * FROM mart.mart_order_fulfillment LIMIT 10;"

# 查看本地数据质量报告
Start-Process ".\data\processed\data_quality_report.html"
```

2.5 数据质量怎样解释

质量报告执行确定性检查：核心表行数、业务键重复、跨表孤儿、时间字段完整性和数值范围。例如 `order_reviews` 允许同一 `order_id` 存在多条评价记录，因此报告中 `duplicate_review_ids` 不应被机械地当作数据库错误；它需要在 MART 建模时通过明确粒度处理。真正需要阻断的情况是订单主键重复、订单商品找不到订单、订单客户找不到客户等。



图 3 实际生成的数据质量报告。核心业务键和跨表完整性检查均有可核对结果。

3. 系统架构与端到端数据流

3.1 分层架构

层	主要技术	职责
交互层	React + TypeScript + Vite	场景入口、会话、流式进度、报告与 Trace
API 层	FastAPI + Pydantic	REST、SSE、参数校验、健康检查、OpenAPI
运行层	RunService + LangGraph	异步运行、状态机、节点编排、Checkpoint
工具层	Tool Registry + AnalyticsService	白名单工具、参数 Schema、只读 SQL
数据层	PostgreSQL 16 + SQLAlchemy	RAW/STAGING/MART、Run/Trace、知识库
检索层	FTS + pgvector + Weighted RRF	关键词与向量候选融合、引用生成
模型层	Ollama qwen2.5:3b	基于工具事实与知识证据进行文本合成
质量层	Evidence Gate + Critic	证据门控、引用覆盖、结构和泄漏检查

3.2 一次问题的调用链

1. 浏览器先 POST /api/v1/sessions 创建会话。
2. 用户消息 POST 到 /api/v1/sessions/{session_id}/messages，API 返回 202 和 run_id。
3. RunService 启动 LangGraph，并把运行状态写入 PostgreSQL。
4. normalize_request 清理输入；route_intent 识别业务意图；build_plan 选择工具计划。
5. run_analytics 只调用注册表中的分析工具，SQL 参数由 Pydantic 模型验证。
6. retrieve_knowledge 执行混合 RAG，得到知识片段和 K-xxx 引用。
7. validate_evidence 确认数据结论、流程建议和引用之间的对应关系。
8. synthesize 把结构化指标与知识证据交给本地模型；模型不可用时使用模板降级。
9. critic 检查输出；finalize 保存最终消息并发布 SSE 事件。
10. 前端通过 /events 接收状态与 token，通过 /trace 查看节点、工具、检索和错误。

3.3 为什么采用 LangGraph

项目需要的不是单轮 prompt，而是有固定顺序、可持久化状态、可观测节点和明确失败边界的业务流程。LangGraph 允许把规范化、路由、规划、分析、检索、证据校验、合成、审查和结束拆成节点；节点输入输出使用 AgentState 传递。这样能单测每个阶段，也能在 Trace 中回答“模型为什么得到这个结论”。当前检查点已落 PostgreSQL，但审批恢复和跨进程可靠任务恢复仍列为后续工作。

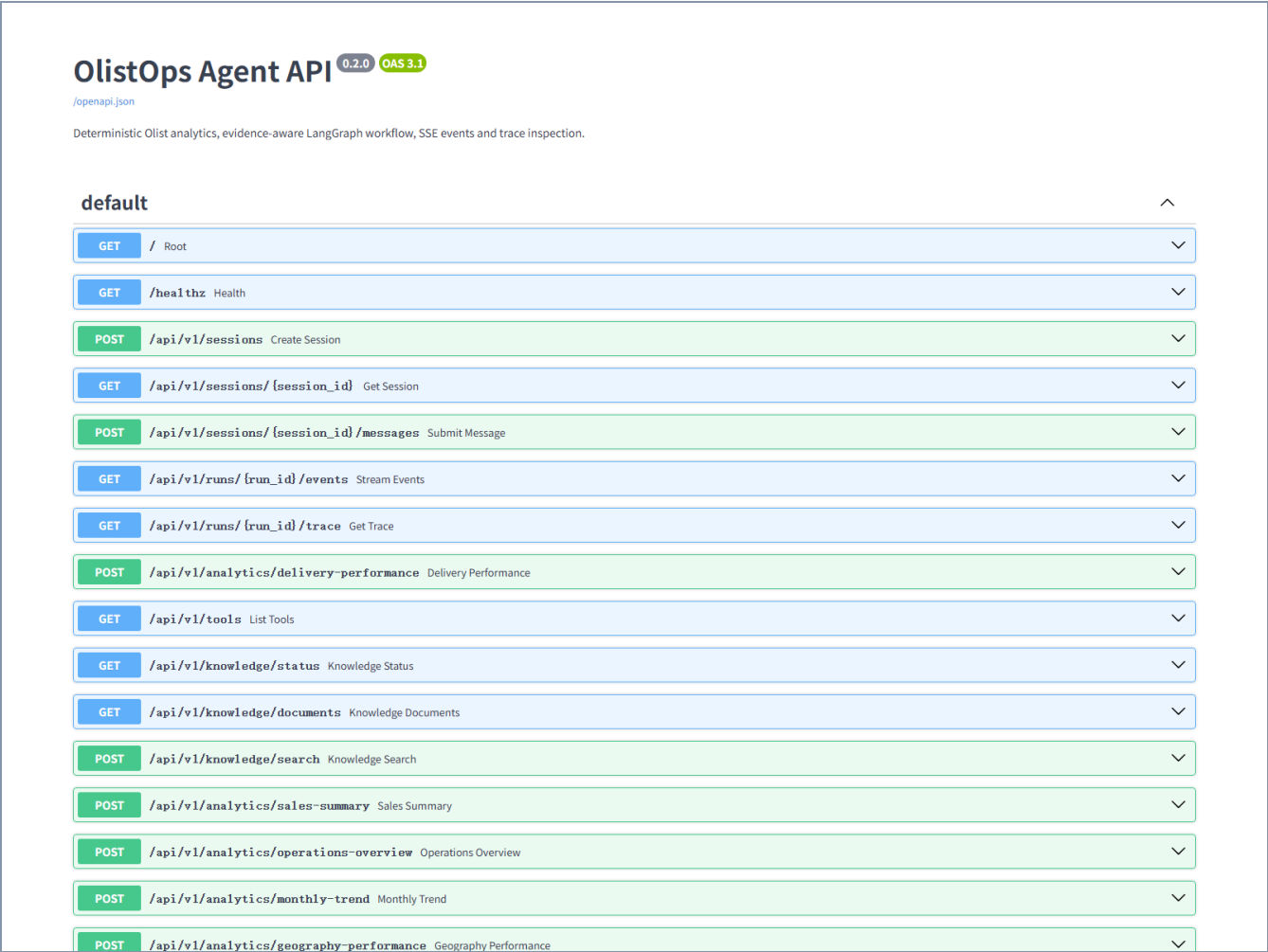


图 4 当前 FastAPI 自动生成的 OpenAPI/Swagger 页面，展示会话、SSE、Trace、知识库和分析接口。

4. Agent 具体能做什么

4.1 十类业务意图

Intent	典型问题	主要工具
operations_overview	整体订单、销售、履约状态如何？	overview + sales + delivery
sales_summary	某时间段 GMV 代理值、客单价如何？	get_sales_summary
delivery_analysis	哪些卖家或品类延迟率高？	get_delivery_performance
trend_analysis	月度订单、销售、履约趋势如何？	get_monthly_trend
seller_diagnostic	一个卖家的综合表现和评论主题？	get_seller_diagnostic
review_analysis	延迟与低分是否相关，差评说了什么？	review tools
geography_analysis	不同州的订单、运费和时效怎样？	get_geography_performance
freight_analysis	运费占比高的地区/品类有哪些？	get_freight_analysis
payment_analysis	支付方式、分期和客单价怎样？	get_payment_analysis
mixed	跨多个域的复合问题	按计划组合白名单工具

4.2 十个只读 Analytics Tools

工具名	作用	关键输入
olist.get_operations_overview	订单状态和经营总览	日期范围
olist.get_sales_summary	订单数、销售额代理值、客单价	日期范围
olist.get_delivery_performance	延迟率、配送天数、分组排名	日期、group_by、top_k
olist.get_monthly_trend	月度经营与履约趋势	日期范围
olist.get_geography_performance	客户州/卖家州表现	日期、维度、top_k
olist.get_freight_analysis	运费及运费占比	日期、group_by、top_k
olist.get_payment_analysis	支付类型、分期、金额	日期、top_k
olist.compare_late_vs_on_time_reviews	延迟与按时订单评分对比	日期范围
olist.search_review_samples	匿名低分评论样本	日期、筛选、limit
olist.get_seller_diagnostic	卖家多指标诊断	seller_id、日期

工具返回的是结构化 JSON，并带有口径、过滤条件和数据新鲜度信息。模型不参与数值计算；其职责是解释工具结果、连接知识证据并组织报告。多商品和多卖家订单先在 MART 中聚合到目标粒度，避免直接 JOIN 明细造成评分或订单数重复。

4.3 建议从这些问题开始

- 分析 2018 年 7 月的配送表现，找出订单量不少于 20 的高延迟卖家与品类。
- 比较延迟订单与按时订单的平均评分，并给出有证据的改进建议。
- 按月查看 2017 年至 2018 年的订单量、销售额代理值、延迟率与平均评分。
- 分析不同客户州的订单量、平均运费、平均配送天数和延迟率。
- 检查支付方式和分期数的金额分布，但不要把支付记录金额简单当作公司财务收入。

5. RAG 到底如何实现、如何使用

5.1 当前知识库是什么

当前数据库有 16 篇项目内模拟知识文档，切分为 60 个 Chunk，其中 60 个已有向量，覆盖率为 100%。内容包括配送异常分级、卖家复盘 SOP、包装发货检查、指标口径、报告证据规范、评论处理和月度趋势规范等。文档源文件位于 data/knowledge_seed。

5.2 入库流水线

- 1. 读取 Markdown 文件，提取标题、来源元数据和正文。
- 2. 按照 Markdown 标题层级维护 heading_path；以约 600 字符为目标切块，尽量不跨语义段。
- 3. 对规范化后的文档和 Chunk 计算 SHA-256，用于幂等更新和变更识别。
- 4. 保存 document、chunk、ordinal、heading_path、token 近似值和 tsvector。
- 5. 调用 EmbeddingProvider 生成 384 维向量；当前默认是 local_hash。
- 6. 将向量写入 pgvector 列，并用 HNSW cosine 索引支持近邻搜索。
- 7. 更新文档状态和 embedding 覆盖率；失败文档保留错误信息，便于重建。

5.3 local_hash 是什么，为什么必须说清楚

local_hash 是一个确定性、离线、无 API Key 的特征哈希向量器。它把规范化 token 和字符 n-gram 哈希到固定维度并归一化，优点是安装即用、结果可重复、单元测试稳定；缺点是它不是经过语义训练的 Embedding 模型，对同义改写、跨语言和复杂语义的泛化能力有限。因此本项目把它定位为工程回退和离线演示 provider，而不是声称它等价于主流语义 Embedding。生产化时应替换成经业务语料验证的 embedding 服务，并保留相同 Provider 接口。

5.4 混合检索流程

分支	实现	擅长	局限
FTS/ 词法	PostgreSQL tsvector + tsquery，并结合查询扩展、词元、n-gram、ILIKE 和标题命中	业务术语、ID、明确关键词、短语	同义表达和隐含语义较弱
Vector	pgvector cosine，相似度为 1 - cosine distance；HNSW 取候选	弱词面匹配、相似片段	当前 local_hash 的语义能力有限
Fusion	按 Weighted Reciprocal Rank Fusion 合并两路排名	不要求两路分数同尺度，容错稳定	权重和 k 需要用评测集校准

```
score(d) = 0.75 / (60 + rank_fts(d))
          + 0.25 / (60 + rank_vector(d))

cosine_similarity(d, q) = 1 - (embedding_d <=> embedding_q)

当前默认：fts_weight=0.75, vector_weight=0.25, rrf_k=60
```

检索时分别扩大 FTS 和 Vector 候选池，再按上述 RRF 分数排序，最后取 top_k。词法分支权重较高，是因为当前向量 provider 是 local_hash；如果切换到高质量语义Embedding，应重新跑 Gold Set 并调整权重。reranker 接口已留出，但当前未启用。



图 5 实时 Hybrid RAG 查询：16 篇文档、60/60 向量覆盖，以及每条结果的 FTS Rank、Vector Rank 与 RRF Score。

5.5 引用、证据门控与注入防护

- 检索结果进入 AgentState 后按顺序分配 K-001、K-002 等引用 ID；报告末尾保留标题和证据摘录，用户可以回溯到具体 Chunk。
- 数据结论必须来自 Analytics Tool；知识片段只能支持流程、制度和解释，不得代替数据证明“某个卖家实际延迟”。
- Evidence Gate 检查是否有工具事实、是否有知识证据、引用是否与检索集合一致；缺少必要证据时应降级或明确声明限制。
- 知识入库和检索阶段标记潜在 prompt injection；进入模型上下文前清理类似“忽略系统指令”“泄露密钥”等文本，知识文档始终按不可信数据处理。
- Critic 检查非空、Markdown 标题、章节结构、引用覆盖、引用有效性和 prompt 泄漏；当前真实运行得分为 1.0，passed=True。

5.6 RAG 怎么实际使用

```
# 查看知识库状态
Invoke-RestMethod http://127.0.0.1:8000/api/v1/knowledge/status

# 直接执行混合检索
$body = @{
    query = "配送延期和包装检查应该如何处理？"
    limit = 5
    mode = "hybrid"
} | ConvertTo-Json
Invoke-RestMethod -Method Post `
    -Uri http://127.0.0.1:8000/api/v1/knowledge/search `
    -ContentType "application/json; charset=utf-8" `
    -Body $body

# 修改 data/knowledge_seed 后重建
python scripts/build_knowledge.py

# 运行检索 Gold Set 评测
python scripts/evaluate_retrieval.py
```

5.7 当前检索评测怎么读

模式	Recall@5	MRR	Top-1	解释
Hybrid	1.0000	1.0000	1.0000	16 个回归问题均在首位命中期望文档
FTS	1.0000	1.0000	1.0000	模拟知识中的关键词区分度较高
Vector	0.7500	0.5521	0.4375	local_hash 能补充召回，但语义泛化有限

这些指标只证明当前 16 条项目内 Gold Case 的回归结果，不代表开放世界或生产语料的通用准确率。下一步应增加同义改写、否定问题、跨语言、相似政策、无答案、注入文本和时间版本冲突用例，并报告 Recall@k、nDCG、引用正确率与端到端答案忠实度。

6. API 如何连接

6.1 接口设计

方法与路径	用途	关键返回
GET /healthz	数据库、模型、RAG 健康状态	status/database/model/rag
POST /api/v1/sessions	创建会话	session_id
GET /api/v1/sessions/{id}	查看会话与消息	session/messages
POST /api/v1/sessions/{id}/messages	提交问题	HTTP 202 + run_id
GET /api/v1/runs/{id}/events	SSE 进度与文本流	event/data
GET /api/v1/runs/{id}/trace	审计运行详情	run/steps/tool_calls/events
GET /api/v1/tools	工具目录与输入 Schema	tool definitions
GET /api/v1/knowledge/status	知识库状态	文档/Chunk/向量/RRF
GET /api/v1/knowledge/documents	知识文档列表	documents
POST /api/v1/knowledge/search	直接检索	rank/score/excerpt
POST /api/v1/analytics/*	直接调用确定性分析	结构化指标

6.2 为什么提交消息返回 202

Agent 可能需要多次 SQL、检索和本地模型推理，不能让一个普通 HTTP 请求长期阻塞。因此消息接口只负责验证输入、创建 Run 并返回 202 Accepted；客户端随后用 run_id 建立SSE 连接。SSE 比 WebSocket 更适合本项目的单向进度流：浏览器原生支持、代理配置简单、事件可按类型消费。需要双向人机审批时，再评估 WebSocket 或持久化 command API。

6.3 完整 PowerShell 调用示例

```
$base = "http://127.0.0.1:8000"

$session = Invoke-RestMethod -Method Post `
    -Uri "$base/api/v1/sessions" `
    -ContentType "application/json; charset=utf-8" `
    -Body (@{ title = "API 演示" } | ConvertTo-Json)

$accepted = Invoke-RestMethod -Method Post `
    -Uri "$base/api/v1/sessions/$(($session.id)/messages" `
    -ContentType "application/json; charset=utf-8" `
    -Body (@{
        content = "分析 2018 年 7 月配送表现并给出证据化建议"
    } | ConvertTo-Json)

$runId = $accepted.run_id
Invoke-RestMethod "$base/api/v1/runs/$runId/trace"
```

6.4 Python 客户端示例

```
import time
import httpx

base = "http://127.0.0.1:8000"
with httpx.Client(base_url=base, timeout=60, trust_env=False) as client:
    session = client.post(
        "/api/v1/sessions", json={"title": "Python client"}
    ).json()
    accepted = client.post(
        f"/api/v1/sessions/{session['id']}/messages",
        json={"content": "分析 2018 年 7 月的配送表现"},
    ).json()
    run_id = accepted["run_id"]
    while True:
        trace = client.get(f"/api/v1/runs/{run_id}/trace").json()
        if trace["run"]["status"] in {"completed", "failed"}:
            break
        time.sleep(0.5)
    print(trace["run"]["status"], trace["run"]["intent"])
```

6.5 接入其他企业系统时怎么做

BI、运营门户或 Notebook 可直接调用 analytics 接口获得确定性 JSON；需要自然语言报告时调用 session/message/events。生产接入应在 API 前增加企业身份提供商、TLS、API Gateway、租户上下文、速率限

制和审计主体；工具层必须根据主体权限过滤数据，RAG 文档检索必须在SQL 中加入 ACL/tenant 条件，不能只在前端隐藏。

7. 一次真实 Agent 运行的证据

字段	本次真实值
Run ID	d38715fa-9bf0-47e5-933d-7e48313efb8d
状态	completed
识别意图	delivery_analysis
模型	ollama:qwen2.5:3b
节点数	9
工具调用数	3
知识引用数	5
Critic	score=1.0; passed=True

本次问题要求分析 2018 年 7 月配送表现、找出高延迟卖家与品类、比较延迟和按时订单评分，并给出改进建议。实际 Trace 顺序为：

```
normalize_request → route_intent → build_plan → run_analytics → retrieve_knowledge →
validate_evidence → synthesize → critic → finalize
```

实际工具调用为：olist.get_delivery_performance; olist.compare_late_vs_on_time_reviews; olist.get_delivery_performance。这些调用参数和结果摘要均保存在 Trace 中。最终报告的经营数字来自 SQL 工具，改进流程由检索到的模拟政策支持。

真实运行也暴露了当前模型表达的一个改进点：模型曾把平均评分差值写成“late delivery rate”。Trace 与工具结果使这类语义错误可被发现；后续应把 Critic 从格式/引用检查扩展到单位、指标名和数值一致性检查。报告不把该错误数值作为业务结论。



图 6 真实 Agent Run 的节点、模型、Critic、引用和最终报告截图。Run ID 可在本机 Trace API 复核。

8. 实现细节：技术面试官通常会追问什么

8.1 路由和计划是不是让 LLM 随便决定

不是。当前 `route_intent` 使用受控规则对业务关键词和组合条件分类，`build_plan` 用 `intent`→步骤/工具映射生成计划。这样做牺牲一部分开放性，换取演示环境中的可重复性、低延迟和低成本。LLM 主要负责最终语言组织。更复杂版本可以让模型生成结构化 Plan，但必须用 JSON Schema 校验、工具 `allowlist`、步数上限和策略引擎二次审批。

8.2 怎样防止模型生成危险 SQL

- Agent 上下文中没有 `execute_sql` 工具；只能选择 `registry` 中注册的十个工具。
- 每个工具有 Pydantic 输入模型，日期、分组维度、`top_k`、`seller_id` 都被类型和枚举约束。
- SQL 写在 `AnalyticsService` 中，业务值通过绑定参数传入；动态标识符只来自代码枚举映射。
- 工具只读取 STAGING/MART；报告和知识库的写入由独立、明确的应用路径负责。
- 生产环境还应使用数据库只读角色、`statement_timeout`、行级安全和资源配额形成纵深防御。

8.3 模型和 API 的关系

FastAPI 不把用户文本直接转发给模型。`RunService` 先执行 workflow；`AnalyticsService` 产生结构化事实，`RetrievalService` 产生知识证据，`Workflow` 再构造受约束 `prompt` 调用 `ModelGateway`。当前 `ModelGateway` 通过 Ollama 的本地 HTTP API 访问 `qwen2.5:3b`；模型列表和健康状态在 `/healthz` 可见。`Gateway` 隔离了模型供应商细节，因此可扩展到兼容 OpenAI API 的企业模型服务，但要增加密钥托管、重试、超时、并发和成本治理。

8.4 为什么同时使用数据库指标和 RAG

信息类型	正确来源	示例
事实与数值	Analytics Tool / MART	2018-07 某卖家的订单数和延迟率
定义与口径	代码、SQL、知识文档	延迟分母、销售额代理值含义
操作流程	RAG 知识	P1/P2/P3 分级、包装复核步骤
自然语言表达	LLM	把事实与流程组织为可读报告

RAG 不适合回答实时聚合数字，SQL 也不适合承载长篇制度和 SOP。两者分工后，既能得到可复算的指标，又能给出有制度出处的建议；`Evidence Gate` 负责防止来源串位。

8.5 Run、Step、Event、Checkpoint 有什么区别

对象	粒度	用途
Run	一次用户消息的完整执行	状态、意图、最终结果、错误
Step	一个 LangGraph 节点	开始/结束、输入输出摘要、耗时
Tool Call	一次工具调用	工具名、验证后参数、结果摘要、错误
Event	面向客户端的时间序列事件	SSE 进度、token、完成/失败
Checkpoint	工作流状态快照	持久化状态与未来恢复基础

9. 测试、评测、审计与安全

9.1 当前验证基线

检查	当前结果	覆盖内容
pytest	27 passed	工具、工作流、安全、Embedding、数据库集成
Ruff	通过	Python 静态规范
Mypy	通过	核心 Python 类型检查
Frontend build	通过	TypeScript/Vite 生产构建
E2E smoke	2 条场景通过	API、数据库、模型、RAG、Agent 端到端
RAG Gold Set	Hybrid 三项均 1.0	16 条项目内检索回归

9.2 关键安全控制

- 输入：Pydantic 校验、长度限制、日期和枚举边界；会话与 Run 使用 UUID。
- 工具：注册表 allowlist、固定 SQL、绑定参数、动态维度映射、结果行数限制。
- 数据：RAW 原始层只读约定，Analytics 主要查询 MART；Trace 对敏感字段做摘要/脱敏。
- 知识：潜在注入标记、上下文清洗、知识视为不可信证据而不是系统指令。
- 输出：Evidence Gate 与 Critic 校验引用集合、章节、泄漏和非空。
- 审计：Run/Step/Tool/Event 持久化，可通过 Trace API 复核。

9.3 仍需补齐的生产安全

- OIDC/OAuth2 登录、RBAC/ABAC、租户隔离与数据库行级安全。
- 文档级与 Chunk 级 ACL，并在检索 SQL 内强制执行。
- TLS、反向代理、CORS 白名单、CSRF 策略、速率限制和 WAF。
- Secrets Manager、密钥轮换、审计日志不可篡改归档和数据保留策略。
- 依赖漏洞扫描、SBOM、镜像签名、备份恢复演练和灾难恢复目标。
- 高风险写操作必须建立独立工具、审批节点、幂等键、补偿动作和最小权限身份。

9.4 指标和因果解释的边界

本项目把 item price 聚合作为销售额代理值，不等同于会计收入、到账金额或利润；支付表可能一单多记录；评价表可能一单多评价；延迟率必须只在同时存在实际与预计送达日期的订单中计算。观察到“延迟订单评分更低”只能说明相关性，不能单凭该数据证明因果。所有跨期趋势还要检查首末月是否完整、促销、节假日和数据截断。

10. 从零部署、运行与排错

10.1 本地组成与端口

组件	默认地址	说明
React Web	http://127.0.0.1:5173	用户工作台
FastAPI	http://127.0.0.1:8000	REST/SSE/Swagger
PostgreSQL	127.0.0.1:55432	业务、知识和审计数据
Ollama	http://127.0.0.1:11434	本地模型服务

10.2 常用命令

```
# 第一次部署：安装依赖、初始化 PostgreSQL、下载/装载数据、建 MART 和知识库
powershell -ExecutionPolicy Bypass -File .\scripts\bootstrap.ps1

# 日常启动与停止
powershell -ExecutionPolicy Bypass -File .\scripts\start.ps1
powershell -ExecutionPolicy Bypass -File .\scripts\stop.ps1

# 健康检查
Invoke-RestMethod http://127.0.0.1:8000/healthz

# 单独重建数据与知识
python scripts/load_data.py
python scripts/data_quality_report.py
python scripts/build_knowledge.py

# 验证
python -m pytest
python -m ruff check .
python -m mypy packages apps scripts
npm --prefix apps/web run build
```

10.3 故障定位顺序

- 1. 先查看 /healthz，区分 API、数据库、模型和 RAG 哪一层异常。
- 2. 检查 .runtime 下的服务日志和 PID 文件，不要只看前端提示。
- 3. 数据库异常时确认 55432 端口、连接串、迁移版本和 pgvector 扩展。
- 4. RAG 无结果时确认 document_count、chunk_count、embedded_chunk_count 和 provider 一致。
- 5. Agent 失败时用 run_id 请求 /trace，找到最后一个失败 Step 或 Tool Call。
- 6. 模型超时但工具成功时，检查 Ollama 模型是否已拉取；必要时验证模板降级路径。
- 7. 数据指标异常时先确认 MART 粒度和日期边界，再检查 RAW，不要直接修改结果表。

10.4 项目目录导航

目录	内容
apps/api	FastAPI 入口、Schema、RunService
apps/web	React/TypeScript 前端
packages/agent_runtime	LangGraph 状态、节点和编排
packages/analytics	分析 Schema、SQL 服务和工具
packages/retrieval	Embedding Provider 与混合检索
packages/model_gateway	Ollama 模型适配
packages/tools	工具注册表
db	迁移、STAGING、MART SQL
data/knowledge_seed	模拟企业知识 Markdown
scripts	部署、装载、评测、证据和报告生成
tests	单元与集成测试
项目导览与使用指南	从零指南、RAG 说明、技术面试细节和本报告

11. 技术取舍、局限与路线图

11.1 已做的取舍

取舍	当前选择	原因	何时升级
数据库	单 PostgreSQL	业务、向量、审计一体，部署简单	数据量或并发显著增长
向量	local_hash 384d	离线可重复、无密钥	接入真实知识和语义查询
融合	Weighted RRF	不同分支分数无需校准	有足够标注数据训练融合器
编排	确定性路由 + LangGraph	稳定、可测、可追踪	场景开放后引入受控 LLM Planner
模型	Ollama qwen2.5:3b	本地隐私和低成本	复杂生成质量或吞吐要求提高
异步	进程内任务 + SSE	本地样例简单	多实例和可靠恢复需要队列
前端	场景化 React UI	降低运营人员提问门槛	接入企业门户和统一身份

11.2 优先路线图

1. P0: 修正指标名称/单位一致性 Critic; 为每个数值建立 tool_result 路径和断言。

2. P0: 引入正式语义 Embedding, 离线对比 recall、nDCG、延迟和资源成本后再切默认。

3. P1: 支持 PDF/DOCX/XLSX/HTML 解析、元数据、版本、有效期、所有者和文档 ACL。

4. P1: 增加 cross-encoder/LLM reranker, 特别处理相似政策和否定查询。

5. P1: 把任务执行迁移到可靠队列, 支持重试、取消、幂等和恢复。

6. P1: 加入 OIDC、RBAC/ABAC、租户隔离、RLS 和不可篡改审计。

7. P2: 建立人工审批节点与恢复协议, 写操作采用专用服务身份和补偿事务。

8. P2: 建设真实标注集、答案忠实度评测、线上反馈和持续回归看板。

11.3 参考开源项目后的吸收方式

项目设计参考了 enterprise-knowledge-rag 对企业知识采集、切块、检索、引用、权限与评测的关注，也参考了 enterprise-workflow-agent-platform 对 workflow、工具治理、运行审计、检查点和人机协同的分层思想。当前实现选择了适合单机演示的最小闭环，没有直接复制对方代码，也没有声称已覆盖其全部平台能力。

参考	地址	本项目吸收点
enterprise-knowledge-rag	https://github.com/smlfy/enterprise-knowledge-rag	企业 RAG 的知识生命周期、混合检索、引用、评测与权限意识
enterprise-workflow-agent-platform	https://github.com/smlfy/enterprise-workflow-agent-platform	workflow 节点、工具注册、运行状态、审计与后续 HITL 路线

12. 技术面试问答速查

项目最核心的工程价值是什么？

把自然语言、确定性经营指标、企业知识和可审计工作流连接起来。重点不是聊天效果，而是数字可复算、建议有引用、执行可追踪、边界可治理。

为什么不用 LLM 直接写 SQL？

生产数据结构复杂，任意 SQL 有注入、越权、性能和口径风险。白名单工具将查询和业务口径固化，并用参数模型约束输入；开放分析可另建只读沙箱和 SQL 审核器。

RAG 为什么同时用 FTS 和向量？

FTS 对 ID、术语和精确关键词可靠，向量补充弱词面匹配；RRF 在不校准分数尺度的情况下融合排名。当前 local_hash 较弱，所以 FTS 权重更高。

如何证明回答没有幻觉？

不能证明“绝不幻觉”，但能缩小和暴露风险：数值仅来自工具、知识引用可回溯、Evidence Gate 控制来源、Critic 检查引用，Trace 保存实际调用。下一步增加数值一致性校验和答案忠实度评测。

pgvector 的索引怎么选？

当前使用 HNSW cosine，适合读多写少、查询延迟优先的小型知识库；规模扩大后应评估 ef_search、m、ef_construction、内存、召回和构建时间，也可对比 IVFFlat。

SSE 与 WebSocket 如何取舍？

当前主要是服务器向浏览器推送运行进度和文本，SSE 更简单。需要实时双向审批、协作控制或高频二进制通信时再使用 WebSocket。

怎样扩展一个新业务场景？

定义指标口径和 MART；实现 Pydantic 输入与 AnalyticsService 查询；注册工具；加入意图路由和计划；补知识文档；增加单元、集成、E2E 与检索评测；最后更新 UI 和文档。

为什么 Gold Set 结果为 1.0 仍不能说 RAG 完美？

只有 16 条、且来自项目模拟文档，存在样本小和关键词明显的问题。它适合回归，不足以估计生产泛化。需要更大、盲测、含 hard negative 与无答案的集合。

如何做企业级权限？

身份在 API 入口解析为主体和租户；工具查询强制 tenant/RLS；知识文档和 Chunk 存 ACL；检索 SQL 先过滤权限再排序；审计记录主体、目的、数据范围和导出。

一次运行失败怎样排查？

从 run_id 的 Trace 定位失败节点，再看 Tool Call 参数、数据库/模型/RAG 健康和服务日志。结构化 Step/Event 比只看最终错误更快找到边界。

附录 A 可复核文件与命令

证据/文档	本地路径
原始实时证据 JSON	项目导览与使用指南/项目介绍报告/actual_evidence.json
六张真实截图	项目导览与使用指南/项目介绍报告/assets
从零导览	项目导览与使用指南/01_从零认识OlistOps_Agent.md
安装使用	项目导览与使用指南/02_从零安装启动与使用.md
RAG 指南	项目导览与使用指南/07_混合RAG与评测指南.md
技术全细节	项目导览与使用指南/技术面试官_项目全细节.md
证据采集脚本	scripts/capture_report_evidence.py
截图脚本	scripts/capture_report_screenshots.py
本报告生成脚本	scripts/generate_project_report.py

```
# 重新采集真实证据（服务需已启动）
python scripts/capture_report_evidence.py
python scripts/capture_report_screenshots.py

# 重新生成 Word、HTML 和 PDF
python scripts/generate_project_report.py
```

附录 B 术语

术语	含义
Agent	能根据目标执行多阶段计划、调用工具并维护状态的软件运行体
RAG	Retrieval-Augmented Generation，检索增强生成
FTS	Full-Text Search，数据库全文检索
Embedding	把文本映射为向量表示的过程
pgvector	PostgreSQL 向量类型、距离运算与索引扩展
RRF	Reciprocal Rank Fusion，倒数排名融合
MART	面向稳定分析口径设计的数据集市
SSE	Server-Sent Events，服务器单向事件流
Evidence Gate	在生成前检查事实与知识证据是否充分的门控
Critic	在输出后执行结构、引用和安全检查的审查节点
HITL	Human in the Loop，人机协同与审批

本报告截至上述证据采集时间反映本地 v0.2.0 实现。代码、数据或配置变化后，应重新运行证据采集、截图和报告生成脚本，以保证数字与截图继续对应当前系统。